

**UCCD 2103**  
**OPERATING SYSTEMS**  
**OCT 2025**  
**ASSIGNMENT**  
**PART B**

| No. | Student ID | Name            | Programme<br>(CN/CS/CT/IA/IG) | Question No. |
|-----|------------|-----------------|-------------------------------|--------------|
| 1   | 2302407    | Chen Pin Yan    | CN                            | 1a & 2a      |
| 2   | 2302663    | Fong Jing Xiang | CN                            | 1b           |
| 3   | 2400897    | Kam Qing Wen    | CN                            | 2b           |
| 4   | 2205586    | Yong Kien Yew   | IA                            | 2b           |

## TABLE OF CONTENTS

| <b>No</b> | <b>Contents</b>                                                       | <b>Page No.</b> |
|-----------|-----------------------------------------------------------------------|-----------------|
| 1         | Question 1 (a)                                                        | <b>3-5</b>      |
| 2         | Question 1 (b)                                                        | <b>6-12</b>     |
| 3         | Question 2 (a)                                                        | <b>13-14</b>    |
| 4         | Question 2 (b)                                                        | <b>15-20</b>    |
| 5         | Appendix A: Content of “e-commerce-solution.c”                        | <b>21-22</b>    |
| 6         | Appendix B: Content of “train-solution.c”                             | <b>23</b>       |
| 7         | Appendix C: Virtual Machine specs that you use to test your programs. | <b>24-27</b>    |
| 8         | Reference                                                             | <b>28</b>       |

## Question 1(a)

1. Compile e-commerce.c into an executable file.

```
chenpinyan@osvm:~/Downloads/Assignment_Part_B_material$ gcc e-commerce.c -o e-commerce -lpthread -lrt
chenpinyan@osvm:~/Downloads/Assignment_Part_B_material$ ./e-commerce
Final stock balance: 1
chenpinyan@osvm:~/Downloads/Assignment_Part_B_material$
```

2. Enable runner.sh to be executable and run the script.

```
chenpinyan@osvm:~/Downloads/Assignment_Part_B_material$ chmod +x runner.sh
chenpinyan@osvm:~/Downloads/Assignment_Part_B_material$ ./runner.sh e-commerce
```

3. The output shown below.

```
chenpinyan@osvm: ~/Downloads/Assignment_Part_B_material
Run no. 461 Final stock balance: 1
Run no. 462 Final stock balance: 1
Run no. 463 Final stock balance: 1
Run no. 464 Final stock balance: 1
Run no. 465 Final stock balance: 0
Run no. 466 Final stock balance: 1
Run no. 467 Final stock balance: 1
Run no. 468 Final stock balance: 1
Run no. 469 Final stock balance: 0
Run no. 470 Final stock balance: 0
Run no. 471 Final stock balance: 1
Run no. 472 Final stock balance: 1
Run no. 473 Final stock balance: 0
Run no. 474 Final stock balance: 0
Run no. 475 Final stock balance: 1
Run no. 476 Final stock balance: 0
Run no. 477 Final stock balance: 0
Run no. 478 Final stock balance: 1
Run no. 479 Final stock balance: 1
Run no. 480 Final stock balance: 0
Run no. 481 Final stock balance: 1
Run no. 482 Final stock balance: 1
Run no. 483 Final stock balance: 1
Run no. 484 Final stock balance: 1
Run no. 485 Final stock balance: 1
Run no. 486 Final stock balance: 1
Run no. 487 Final stock balance: 1
Run no. 488 Final stock balance: 0
Run no. 489 Final stock balance: 1
Run no. 490 Final stock balance: 0
Run no. 491 Final stock balance: 1
Run no. 492 Final stock balance: 1
Run no. 493 Final stock balance: 1
Run no. 494 Final stock balance: 0
Run no. 495 Final stock balance: 1
Run no. 496 Final stock balance: 1
Run no. 497 Final stock balance: 1
Run no. 498 Final stock balance: 1
Run no. 499 Final stock balance: 1
chenpinyan@osvm:~/Downloads/Assignment_Part_B_material$
```

**Timeline Table: Race Condition Leading to Incorrect Balance**

| Time(s) | Thread | Execution                        | Local Variable (t) | Shared Variable (balance) | Observation                                                                                                        |
|---------|--------|----------------------------------|--------------------|---------------------------|--------------------------------------------------------------------------------------------------------------------|
| T=0.0   | OS     | Initial state                    |                    | 2                         | Stock is 2 units.                                                                                                  |
| T=0.1   | t0     | int t = balance;                 | 2                  | 2                         | t0 reads the initial balance of 2.                                                                                 |
| T=0.1   | t0     | sleep(0.1); (Start sleep)        | 2                  | 2                         | t0 is suspended by the OS for I/O latency.                                                                         |
| T=0.1   | OS     | Preemption: Switch to t1.        |                    | 2                         |                                                                                                                    |
| T=0.2   | t1     | int t = balance;                 | 2                  | 2                         | t1 also reads the outdated initial balance of 2.                                                                   |
| T=0.2   | t1     | sleep(0.1); (Start sleep)        | 2                  | 2                         | t1 is suspended by the OS for I/O latency.                                                                         |
| T=0.3   | t0     | sleep(0.1); (End sleep, resumes) | 2                  | 2                         |                                                                                                                    |
| T=0.4   | t0     | if(balance > 0) -> True          | 2                  | 2                         | t0 proceeds to deduct.                                                                                             |
| T=0.4   | t0     | balance = t - 1;                 | 2                  | 1                         | t0 updates balance to 2-1=1                                                                                        |
| T=0.4   | OS     | Preemption: Switch to t1.        |                    | 1                         |                                                                                                                    |
| T=0.5   | t1     | sleep(0.1); (End sleep, resumes) | 2                  | 1                         |                                                                                                                    |
| T=0.6   | t1     | if(balance > 0) -> True          | 2                  | 1                         | t1 proceeds to deduct.                                                                                             |
| T=0.7   | t1     | balance = t - 1;                 | 2                  | 1                         | <b>ERROR:</b> t1 calculates $2 - 1 = 1$ (using its outdated local $t=2$ ) and overwrites the shared balance with 1 |

### **Observations:**

When running the e-commerce program where two customers simultaneously order the same product, the final stock balance was sometimes incorrect. Specifically, although the initial stock was 2 units and two customers each ordered 1 unit, the final stock occasionally showed 1 instead of 0. This inconsistency indicates that one of the orders did not correctly deduct the stock, leading to an over-selling situation.

The root cause of this problem lies in the way the two threads access the shared stock variable. Both threads first read the stock balance with a simulated delay of 0.1 seconds to mimic I/O latency. Because there is no synchronization mechanism, both threads may read the stock balance before either thread has updated it. Consequently, both threads think the stock is still 2 units and proceed to place their orders, each deducting 1 unit. However, since both deductions are based on the original stock read, one of the deductions effectively overwrites the other, causing the final stock count to be inaccurate.

Furthermore, when running the program repeatedly (for example, looping 500 times with a runner script), this race condition becomes more evident. Some runs show the correct final stock of 0, but many shows incorrect values due to the timing overlap between the threads. This unpredictable interleaving of thread execution causes the critical section the reading and updating of stock to be vulnerable to race conditions.

**Race condition** occurs when two or more threads or processes access and manipulate shared data concurrently without proper synchronization, and the outcome depends on the unpredictable timing (or “race”) of their execution [4]. Because the threads interleave in an uncontrolled manner, this leads to inconsistent or incorrect behaviour.

Overall, the program demonstrates a classic concurrency problem that is the lack of proper synchronization results in a race condition, where multiple threads compete to access and modify shared data without coordination. This leads to inconsistent and erroneous behaviour that undermines the correctness of the stock management in the e-commerce scenario.

## **Question 1(b)**

### **1. Problem Analysis & Solution Approach**

In part 1(a) the timeline analysis showed that both customer threads read same outdated balance value before either thread performed the deduction. This has caused an incorrect final stock balance result 1 instead of 0, showing a race condition occur. The original program lacked synchronization mechanism, so both threads could execute the read–delay–deduct sequence concurrently. To avoid the issue, the critical steps must be executed in a way that lets only one thread access and modify the stock at once. The parts of the process that require synchronization are:

- Read stock
- Wait (simulate I/O delay)
- Check stock
- Deduct stock

The solution is to use a binary semaphore as a mutex so that only one customer thread can enter the critical section at a time. This stops both threads from reading the outdated balance and ensure correct, consistent results, solve the concurrency problem.

### **2. Solution Implementation Using Semaphore**

The corrected program uses a POSIX semaphore initialized 1 to implement a binary semaphore (mutex) for protecting access. The below show key changes were made.

- Declaring a semaphore:

```
sem_t stock_sem;
```

- Initializing the semaphore with value 1 (binary semaphore):

```
sem_init(&stock_sem, 0, 1);
```

- Wrapping the critical section inside sem\_wait and sem\_post:

```
sem_wait(&stock_sem); //Lock  
int t = balance;  
sleep(0.1);  
if (balance > 0)  
    balance = t - 1;  
sem_post(&stock_sem); //Unlock
```

This ensures that the entire read–sleep–deduct sequence is executed by only one thread at a time. The second thread must wait until the first thread completes its update.

### **3. Explanation of Semaphore Implementation**

A semaphore acts like a gatekeeper to the critical section:

1. `sem_wait(&stock_sem)`
  - Decreases the semaphore value from **1** → **0**
  - If the value is already **0**, the thread **blocks and waits**
2. `sem_post(&stock_sem)`
  - Increases the semaphore value from **0** → **1**
  - Signals that the next thread may proceed

In the original code, the two threads ran almost same time. Since both were delayed by `sleep(0.1)`, they each read `balance = 2` before any deduction occurred. With the implementation of semaphore, only one thread is allowed to enter the critical section at a time. This ensures that one thread will completes the full read–sleep–deduct sequence before another can begin. While the first thread is running, the second thread must wait until the critical section becomes free. This prevents the second thread from reading an outdated balance, fixing the overwrite problem in Part 1(a).

### **4. Comparison with Original Code**

| Aspect                    | Original Code                         | Corrected Code                                       |
|---------------------------|---------------------------------------|------------------------------------------------------|
| Synchronization           | No synchronization                    | Uses <code>sem_wait</code> and <code>sem_post</code> |
| Thread Safety             | Unsafe due to race condition present  | Safe and race-free                                   |
| Critical Section Handling | Not protected                         | Protected with binary semaphore                      |
| Behaviour                 | Inconsistent: final balance = 0 or 1  | Consistent: final balance = 0                        |
| Execution Order           | Uncontrolled interleaving             | Forced strictly sequential access                    |
| Shared Variable Access    | Both threads read same outdated value | Each thread reads updated value                      |

## 5. Testing and Results

### Original Program Results

Multiple runs produced inconsistent results.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>fong@fongjingxiang-virtualbox:~/Downloads\$ chmod +x runner.sh fong@fongjingxiang-virtualbox:~/Downloads\$ ./runner.sh e-commerce Run no. 0 Final stock balance: 1 Run no. 1 Final stock balance: 1 Run no. 2 Final stock balance: 1 Run no. 3 Final stock balance: 1 Run no. 4 Final stock balance: 0 Run no. 5 Final stock balance: 1 Run no. 6 Final stock balance: 0 Run no. 7 Final stock balance: 1 Run no. 8 Final stock balance: 1 Run no. 9 Final stock balance: 1 Run no. 10 Final stock balance: 1 Run no. 11 Final stock balance: 1 Run no. 12 Final stock balance: 1 Run no. 13 Final stock balance: 0 Run no. 14 Final stock balance: 1 Run no. 15 Final stock balance: 1 Run no. 16 Final stock balance: 1 Run no. 17 Final stock balance: 1 Run no. 18 Final stock balance: 0 Run no. 19 Final stock balance: 1</pre> | <pre>Run no. 475 Final stock balance: 0 Run no. 476 Final stock balance: 1 Run no. 477 Final stock balance: 1 Run no. 478 Final stock balance: 1 Run no. 479 Final stock balance: 1 Run no. 480 Final stock balance: 1 Run no. 481 Final stock balance: 0 Run no. 482 Final stock balance: 1 Run no. 483 Final stock balance: 1 Run no. 484 Final stock balance: 1 Run no. 485 Final stock balance: 1 Run no. 486 Final stock balance: 1 Run no. 487 Final stock balance: 1 Run no. 488 Final stock balance: 1 Run no. 489 Final stock balance: 1 Run no. 490 Final stock balance: 1 Run no. 491 Final stock balance: 1 Run no. 492 Final stock balance: 1 Run no. 493 Final stock balance: 1 Run no. 494 Final stock balance: 1 Run no. 495 Final stock balance: 1 Run no. 496 Final stock balance: 1 Run no. 497 Final stock balance: 1 Run no. 498 Final stock balance: 1 Run no. 499 Final stock balance: 1</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

This confirmed the race condition occur in Part 1(a).

### Corrected Program Results

Every test produced a consistent final stock balance of zero.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>fong@fongjingxiang-virtualbox:~/Downloads\$ ./runner.sh e-commerce-solution Run no. 0 Final stock balance: 0 Run no. 1 Final stock balance: 0 Run no. 2 Final stock balance: 0 Run no. 3 Final stock balance: 0 Run no. 4 Final stock balance: 0 Run no. 5 Final stock balance: 0 Run no. 6 Final stock balance: 0 Run no. 7 Final stock balance: 0 Run no. 8 Final stock balance: 0 Run no. 9 Final stock balance: 0 Run no. 10 Final stock balance: 0 Run no. 11 Final stock balance: 0 Run no. 12 Final stock balance: 0 Run no. 13 Final stock balance: 0 Run no. 14 Final stock balance: 0 Run no. 15 Final stock balance: 0 Run no. 16 Final stock balance: 0 Run no. 17 Final stock balance: 0 Run no. 18 Final stock balance: 0 Run no. 19 Final stock balance: 0 Run no. 20 Final stock balance: 0 Run no. 21 Final stock balance: 0 Run no. 22 Final stock balance: 0 Run no. 23 Final stock balance: 0</pre> | <pre>Run no. 475 Final stock balance: 0 Run no. 476 Final stock balance: 0 Run no. 477 Final stock balance: 0 Run no. 478 Final stock balance: 0 Run no. 479 Final stock balance: 0 Run no. 480 Final stock balance: 0 Run no. 481 Final stock balance: 0 Run no. 482 Final stock balance: 0 Run no. 483 Final stock balance: 0 Run no. 484 Final stock balance: 0 Run no. 485 Final stock balance: 0 Run no. 486 Final stock balance: 0 Run no. 487 Final stock balance: 0 Run no. 488 Final stock balance: 0 Run no. 489 Final stock balance: 0 Run no. 490 Final stock balance: 0 Run no. 491 Final stock balance: 0 Run no. 492 Final stock balance: 0 Run no. 493 Final stock balance: 0 Run no. 494 Final stock balance: 0 Run no. 495 Final stock balance: 0 Run no. 496 Final stock balance: 0 Run no. 497 Final stock balance: 0 Run no. 498 Final stock balance: 0 Run no. 499 Final stock balance: 0</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

The semaphore removed the race condition and ensured correct stock deduction across all executions.

## 6. Timeline Tables for Correct Execution

### Variant A — customer 0 executes first

| Time(s) | Thread 0<br>(customer_0)                       | Thread 1<br>(customer_1)      | Semaphore Value | Balance | Observation                                     |
|---------|------------------------------------------------|-------------------------------|-----------------|---------|-------------------------------------------------|
| T=0.0   | Program start                                  | Program start                 | 1               | 2       | Initial state: stock = 2, semaphore = 1         |
| T=0.1   | sem_wait(&stock_sem) → enters Critical Section | —                             | 0               | 2       | t0 acquires semaphore, t1 cannot enter.         |
| T=0.2   | int t = balance; → t = 2                       | BLOCKED (waiting at sem_wait) | 0               | 2       | t0 reads balance into local t.                  |
| T=0.3   | sleep(0.1); (start)                            | BLOCKED                       | 0               | 2       | t0 simulates I/O delay while holding semaphore. |
| T=0.4   | sleep(0.1); (end, resumes)                     | BLOCKED                       | 0               | 2       | t0 resumes after delay.                         |
| T=0.5   | if (balance > 0) → TRUE                        | BLOCKED                       | 0               | 2       | Condition true, t0 will deduct.                 |
| T=0.6   | balance = t - 1; → balance becomes 1           | BLOCKED                       | 0               | 1       | t0 updates shared balance from 2 → 1            |
| T=0.7   | sem_post(&stock_sem) → exit CS                 | BLOCKED                       | 1               | 1       | t0 releases semaphore, t1 can proceed.          |

|       |                                  |                                                      |   |   |                                                    |
|-------|----------------------------------|------------------------------------------------------|---|---|----------------------------------------------------|
| T=0.8 | Exits critical section / returns | sem_wait(&stock_semaphore) → enters Critical Session | 0 | 1 | t1 acquires semaphore and enters Critical Session. |
| T=0.9 | —                                | int t = balance; → t = 1                             | 0 | 1 | t1 reads updated balance into local t              |
| T=1.0 | —                                | sleep(0.1); (start)                                  | 0 | 1 | t1 simulates I/O delay while in Critical Session.  |
| T=1.1 | —                                | sleep(0.1); (end, resumes)                           | 0 | 1 | t1 resumes after delay                             |
| T=1.2 | —                                | if (balance > 0) → TRUE                              | 0 | 1 | Condition true, t1 will deduct.                    |
| T=1.3 | —                                | balance = t - 1; → balance becomes 0                 | 0 | 0 | t1 updates shared balance from 1 → 0               |
| T=1.4 | —                                | sem_post(&stock_semaphore) → exit Critical Session   | 1 | 0 | t1 releases semaphore                              |
| T=1.5 | Threads terminate                | Threads terminate                                    | 1 | 0 | Final state: balance = 0                           |

**Variant B — customer 1 executes first**

| Time(s) | Thread 0 (customer_0)                             | Thread 1 (customer_1)                             | Semaphore Value | Balance | Remark                                             |
|---------|---------------------------------------------------|---------------------------------------------------|-----------------|---------|----------------------------------------------------|
| T=0.0   | Program start                                     | Program start                                     | 1               | 2       | Initial state: stock = 2, semaphore = 1            |
| T=0.1   | —                                                 | sem_wait(&stock_sem)<br>→ enters Critical Session | 0               | 2       | t1 acquires semaphore, t0 cannot enter.            |
| T=0.2   | BLOCKED (waiting at sem_wait)                     | int t = balance; → t = 2                          | 0               | 2       | t1 reads balance into local t.                     |
| T=0.3   | BLOCKED                                           | sleep(0.1); (start)                               | 0               | 2       | t1 simulates I/O delay while holding semaphore.    |
| T=0.4   | BLOCKED                                           | sleep(0.1); (end, resumes)                        | 0               | 2       | t1 resumes after delay.                            |
| T=0.5   | BLOCKED                                           | if (balance > 0) → TRUE                           | 0               | 2       | Condition true, t1 will deduct.                    |
| T=0.6   | 1 BLOCKED                                         | balance = t - 1; → balance becomes 1              | 0               | 1       | t1 updates shared balance from 2 → 1               |
| T=0.7   | BLOCKED                                           | sem_post(&stock_sem)<br>→ exit CS                 | 1               | 1       | t1 releases semaphore, t0 can proceed.             |
| T=0.8   | sem_wait(&stock_sem)<br>→ enters Critical Session | Exits critical section / returns                  | 0               | 1       | t0 acquires semaphore and enters Critical Session. |

|       |                                              |                   |   |   |                                                   |
|-------|----------------------------------------------|-------------------|---|---|---------------------------------------------------|
| T=0.9 | int t = balance; → t = 1                     | —                 | 0 | 1 | t0 reads updated balance into local t             |
| T=1.0 | sleep(0.1); (start)                          | —                 | 0 | 1 | t0 simulates I/O delay while in Critical Session. |
| T=1.1 | sleep(0.1); (end, resumes)                   | —                 | 0 | 1 | t0 resumes after delay                            |
| T=1.2 | if (balance > 0) → TRUE                      | —                 | 0 | 1 | Condition true, t0 will deduct.                   |
| T=1.3 | balance = t - 1; → balance becomes 0         | —                 | 0 | 0 | t0 updates shared balance from 1 → 0              |
| T=1.4 | sem_post(&stock_sem) → exit Critical Session | —                 | 1 | 0 | t0 releases semaphore                             |
| T=1.5 | Threads terminate                            | Threads terminate | 1 | 0 | Final state: balance = 0                          |

## **7. Conclusion**

The original program had a race condition because of both threads were reading and changing the balance at the same time, causing wrong results. Adding a binary semaphore to fix this by allowing only one thread to run the read–sleep–deduct steps at a time. This stopped the threads from affecting with each other and removed the error seen in Part 1(a). With the fix, the final stock balance always came out correctly as 0, showing that semaphores are useful for controlling access in multithreaded programs.

## Question 2(a)

The command to open original code:

```
chenpinyan@osvm:~$ cd '/home/chenpinyan/Downloads/Assignment_Part_B_material'  
chenpinyan@osvm:~/Downloads/Assignment_Part_B_material$ gcc train.c -o train -lpthread -lrt  
chenpinyan@osvm:~/Downloads/Assignment_Part_B_material$ ./train
```

The original code content with error:

```
The railway gate is closed  
  
The train is crossing  
  
The cars are crossing the railway  
  
The railway gate is opened
```

**Timeline Table: Incorrect interleaving between the train thread and the traffic thread**

| Time(s) | Thread | Execution                                                                           | Output                            |
|---------|--------|-------------------------------------------------------------------------------------|-----------------------------------|
| T=0.0   | OS     | pthread_create for car (t0) and train (t1).                                         |                                   |
| T=0.1   | Train  | Enters while(1) loop.                                                               |                                   |
| T=0.2   | Train  | printf("\n The railway gate is closed\n");                                          | The railway gate is closed        |
| T=0.3   | Train  | printf("\n The train is crossing\n");                                               | The train is crossing             |
| T=0.4   | Train  | sleep(1); (Train crossing)                                                          |                                   |
| T=0.5   | OS     | Preemption: The train is put to sleep, allowing the OS to switch to the car thread. |                                   |
| T=0.6   | Car    | Enters while(1) loop.                                                               |                                   |
| T=0.7   | Car    | printf("\n The cars are crossing the railway\n");                                   | The cars are crossing the railway |
| T=0.8   | Car    | sleep(1); (Car crossing sleep)                                                      |                                   |
| T=1.3   | OS     | Preemption: The car is put to sleep.                                                |                                   |
| T=1.4   | Train  | Resumes execution (1 second sleep finishes).                                        |                                   |
| T=1.5   | Train  | printf("\n The railway gate is opened\n");                                          | The railway gate is opened        |
| T=1.6   | Train  | sleep(5); (Waiting for next train)                                                  |                                   |

### Observations:

While run the train program, the printed output often appears in an incorrect and unsafe order. Specifically, the message “The cars are crossing the railway”. Sometimes appears while the train is still crossing and the gate is reported as closed.

The problem occurs because both the train thread and the car thread run concurrently without any form of synchronization. There is no semaphore or locking mechanism to prevent the car thread from printing its message during the train’s critical section (when the gate is closed and the train is crossing). As a result, their outputs interleave unpredictably, creating a race condition.

This race condition causes an unsafe execution sequence where cars appear to pass through the railway even though the gate is closed and the train is currently crossing.

### Problem of the program:

- No synchronization between threads

```
9  void* train(void* arg)
10 {
11     while(1){
12         printf("\n The railway gate is closed\n");
13
14         //critical section
15         printf("\n The train is crossing\n");
16         sleep(1); //train crossing for 1 second
17
18         printf("\n The railway gate is opened\n");
19         sleep(5); //the next train will arrive every 5 seconds
20     }
21 }
22
23 void* car(void* arg)
24 {
25     while(1){
26         printf("\n The cars are crossing the railway\n");
27         sleep(1);
28     }
29 }
```

By observing the figure above, we can identify that, the train thread simulates the gate closing, train crossing, and gate opening. The car thread just prints that cars are crossing every second and there is no coordination or control (like a semaphore or mutex) to prevent cars crossing while the train is crossing.

## Question 2(b)

### 1. Proposed Solution

According to above observation between train and car thread, it shows a race condition problem appear in this process, which means the execution becomes random that both threads run at the same time without order sequences.

To solve this, it is recommended to implement **POSIX semaphore** a synchronization technique because semaphore involve a counter and a blocking mechanism that can control multiple threads access to use the road ensuring only one thread enters the crossing at a time.

### 2. Implementation and Explanation

Key concepts: An initial counter(`sem_init()`), two operations (`sem_wait()`, `sem_post()`)

#### Step1: Initialization

| kamqingwen@osvm: ~/Downloads                                                                                                                  | kamqingwen@osvm: ~/Downloads                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>GNU nano 6.2 train-solu #include &lt;unistd.h&gt;  sem_t gate; //Global variable, 1=road free, 0=road busy  void* train(void* arg)</pre> | <pre>GNU nano 6.2 train-solutio int thread0_id=0, thread1_id=1;  sem_init(&amp;gate, 0, 1); //Initialise semaphore to 1  pthread_t t0,t1;</pre> |

**sem\_t gate:** Global variable, named gate, represents the semaphore control, not the railway gate state.

- gate = 1 means the road is free, anyone may request to enter.
- gate = 0 means the road is busy, no one else can enter.

**sem\_init(&gate, 0, 1):** Initialize semaphore, 0 means semaphore shared between threads in one process, 1 means initially road is free. If 0 instead of 1, the road is busy from the start, cars cannot enter, train also cannot enter, the program stuck.

#### Step2: Modify train thread

```
kamqingwen@osvm: ~/Downloads
GNU nano 6.2 train-solution.c *

void* train(void* arg)
{
    while(1){
        sem_wait(&gate); //Decrease gate to use road
        printf("\n The railway gate is closed\n");

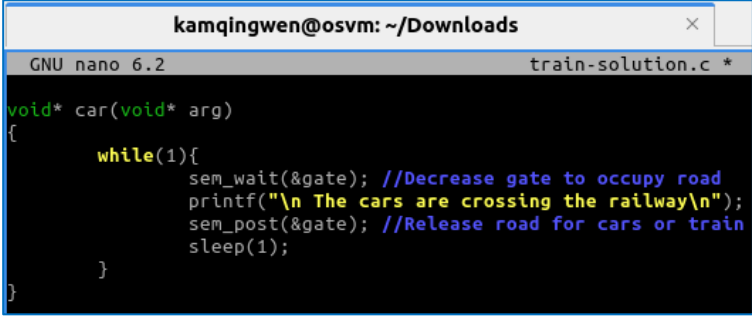
        printf("\n The train is crossing\n");
        sleep(1);

        printf("\n The railway gate is opened\n");
        sem_post(&gate); //Increase gate to release road
        sleep(5);
    }
}
```

Add **sem\_wait(&gate)** and **sem\_post(&gate)** according to the position in above figure. At the beginning, the train arrives and tries to use the road, but the train thread slower than the car thread get CPU, the cars' **sem\_wait** turn 0 and block the train thread to prevent train crossing, so the train need to wait for cars finish. When the cars finish, the cars' **sem\_post** change gate becomes 1(road free) and unblocks the train. The train tries to enter again and is successful, the **sem\_wait** change gate decreases to 0, blocking the cars enter the semaphore. The train gets into the critical section that is crossing the road for 1 second, then after crossing the road, **sem\_post** calls to increase semaphore value become 1 and the road free to cars. The semaphore sleep 5 seconds until next train arrives.

According to the original code, the program prints "gate is closed" and run into critical section means the **sem\_wait(&gate)** command need to actual operate first then show the print message. However, the **sem\_post(&gate)** command should run after print message because if the gate opens first, the cars may cross before the print message "gate is opened" and the message will become delayed. Besides, during the 5 seconds of waiting, the cars can cross the railway which means the gate is open, so that *sleep(5)* doesn't include in the semaphore process.

### Step3: Modify car thread



```

kamqingwen@osvm: ~/Downloads
GNU nano 6.2 train-solution.c *
void* car(void* arg)
{
    while(1){
        sem_wait(&gate); //Decrease gate to occupy road
        printf("\n The cars are crossing the railway\n");
        sem_post(&gate); //Release road for cars or train
        sleep(1);
    }
}
  
```

Add **sem\_wait(&gate)** and **sem\_post(&gate)** according to the position in above figure. At the beginning, the car tries to cross the road (acquire semaphore) and the car successful enter because the cars do small work with 4 lines and loop faster than train loop. The cars' **sem\_wait** change gate 1 to 0(road busy) and block the train trying to use the road. The car is crossing the road, which running the critical section. After the car crossing, its **sem\_post** change gate becomes 1 and unblocks other threads, allowing the next car or the train to acquire it.

The **sleep(1)** was written after **sem\_post** because the question states "prevent the infinite while loop from running too fast" which is for the thread not for the car crossing time, so that it doesn't include in the critical section. As the car immediately **sem\_post** and the train wait 5 seconds before arriving, multiple cars could pass in the car thread, thus the cars print about 5 times in the above output figure.

#### Step4: Destroy semaphore

```
kamqingwen@osvm: ~/Downloads
GNU nano 6.2
pthread_join(t1,NULL);

sem_destroy(&gate); //Clean up semaphore

return 0;
}
```

**sem\_destroy(&gate):** Clean up the semaphore before it ends. However, this line will never run in the current program because the join() calls block forever.

### 3. Test Result

#### a) Original Code Result (train.c)

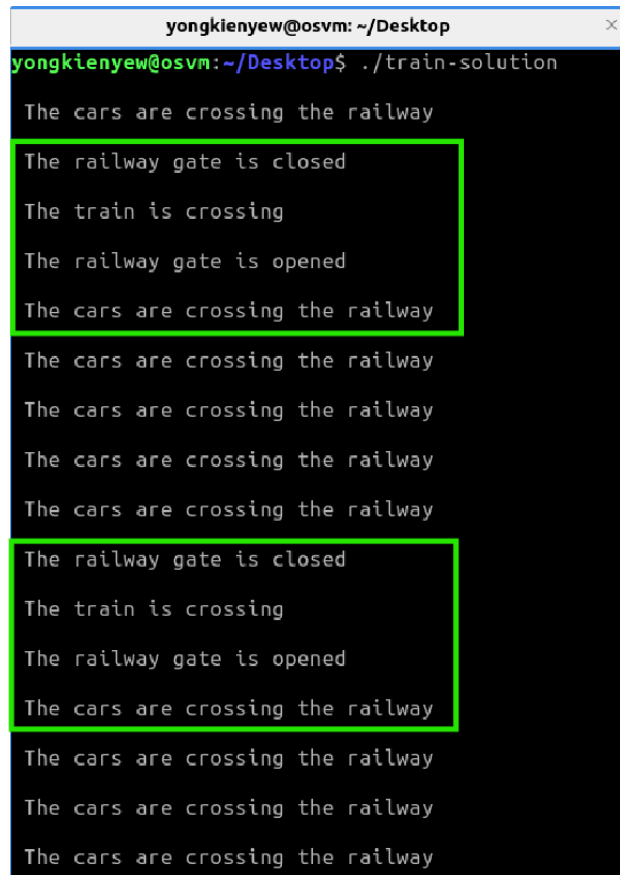
Incorrect order of messages being printed out due to no synchronization being implemented.

However, it sometimes had corrected order of message being printed out. This is because race condition happened when no rule is implemented to stop both these threads to run at the same time.

```
yongkienyew@osvm: ~/Desktop
yongkienyew@osvm:~/Desktop$ ./train
The railway gate is closed
The train is crossing
The cars are crossing the railway
The railway gate is opened
The cars are crossing the railway
The cars are crossing the railway
The cars are crossing the railway
The cars are crossing the railway
The railway gate is closed
The train is crossing
The railway gate is opened
The cars are crossing the railway
The cars are crossing the railway
The cars are crossing the railway
```

#### b) Fixed Version Result (train-solution.c)

After implementing synchronization using POSIX semaphore, the order of the messages has been corrected.



```
yongkienyew@osvm: ~/Desktop
yongkienyew@osvm:~/Desktop$ ./train-solution

The cars are crossing the railway
The railway gate is closed
The train is crossing
The railway gate is opened
The cars are crossing the railway
The cars are crossing the railway
The cars are crossing the railway
The cars are crossing the railway
The cars are crossing the railway
The railway gate is closed
The train is crossing
The railway gate is opened
The cars are crossing the railway
The cars are crossing the railway
The cars are crossing the railway
The cars are crossing the railway
```

#### 4. Timeline Table (with solution code)

This timeline table show the full iterations of the train-solution.c with its own designated infinite loop. This timeline table only shows the starting of the program with the first full iterations.

| Time(s) | Thread | Execution                                        | Output                            | Remarks                                 |
|---------|--------|--------------------------------------------------|-----------------------------------|-----------------------------------------|
| 0.0     | OS     | sem_init(&gate,0,1)                              |                                   | sem_init set the gate as opened.        |
| 0.1     | OS     | pthread_create for car(t0) and train(t1).        |                                   |                                         |
| 0.11    | car    | sem_wait(&gate)                                  |                                   | Decrement the gate's value from 1 to 0. |
| 0.12    | car    | printf("\n The cars are crossing the railway\n") | The cars are crossing the railway |                                         |
| 0.13    | car    | sem_post(&gate)                                  |                                   | Increment the gate's value from 0 to 1. |
| 0.14    | car    | sleep(1)                                         |                                   | The car crossing.                       |

|      |       |                                                  |                                   |                                                                                                                                                                                                              |
|------|-------|--------------------------------------------------|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.15 | OS    |                                                  |                                   | Car is put into sleep, allowing the train thread to run.                                                                                                                                                     |
| 0.16 | train | sem_wait(&gate)                                  |                                   | Decrement the gate's value from 1 to 0.                                                                                                                                                                      |
| 0.17 | train | printf("\n The railway gate is closed \n")       | The railway gate is closed        |                                                                                                                                                                                                              |
| 0.18 | train | printf("\n The train is crossing \n")            | The train is crossing             |                                                                                                                                                                                                              |
| 0.19 | train | sleep(1)                                         |                                   | The train is crossing.                                                                                                                                                                                       |
| 1.19 | train | printf("\n The railway gate is opened \n")       | The railway gate is opened        | This critical session is locked by sem_wait(&gate) in T = 0.16 for train thread to execute, therefore the car thread is unable to execute even though the train thread is sleeping for 1 second in T = 0.19. |
| 1.2  | train | sem_post(&gate)                                  |                                   | Increment the gate's value from 0 to 1.                                                                                                                                                                      |
| 1.21 | train | sleep(5)                                         |                                   | Waiting for the next train that arrived in the next 5 second.                                                                                                                                                |
| 1.22 | OS    |                                                  |                                   | Train is put into sleep, allowing the car thread to run.                                                                                                                                                     |
| 1.23 | car   | sem_wait(&gate)                                  |                                   | Decrement the gate's value from 1 to 0.                                                                                                                                                                      |
| 1.24 | car   | printf("\n The cars are crossing the railway\n") | The cars are crossing the railway | Due to the infinite while loop in car function and the train thread is still sleeping for 5 second. The car function will loop itself until the train thread is awoken.                                      |
| 1.25 | car   | sem_post(&gate)                                  |                                   | Increment the gate's value from 0 to 1.                                                                                                                                                                      |
| 1.26 | car   | sleep(1)                                         |                                   | The car crossing.                                                                                                                                                                                            |
| 2.27 | car   | sem_wait(&gate)                                  |                                   | Decrement the gate's value from 1 to 0.                                                                                                                                                                      |
| 2.28 | car   | printf("\n The cars are crossing the railway\n") | The cars are crossing the railway |                                                                                                                                                                                                              |
| 2.29 | car   | sem_post(&gate)                                  |                                   | Increment the gate's value from 0 to 1.                                                                                                                                                                      |

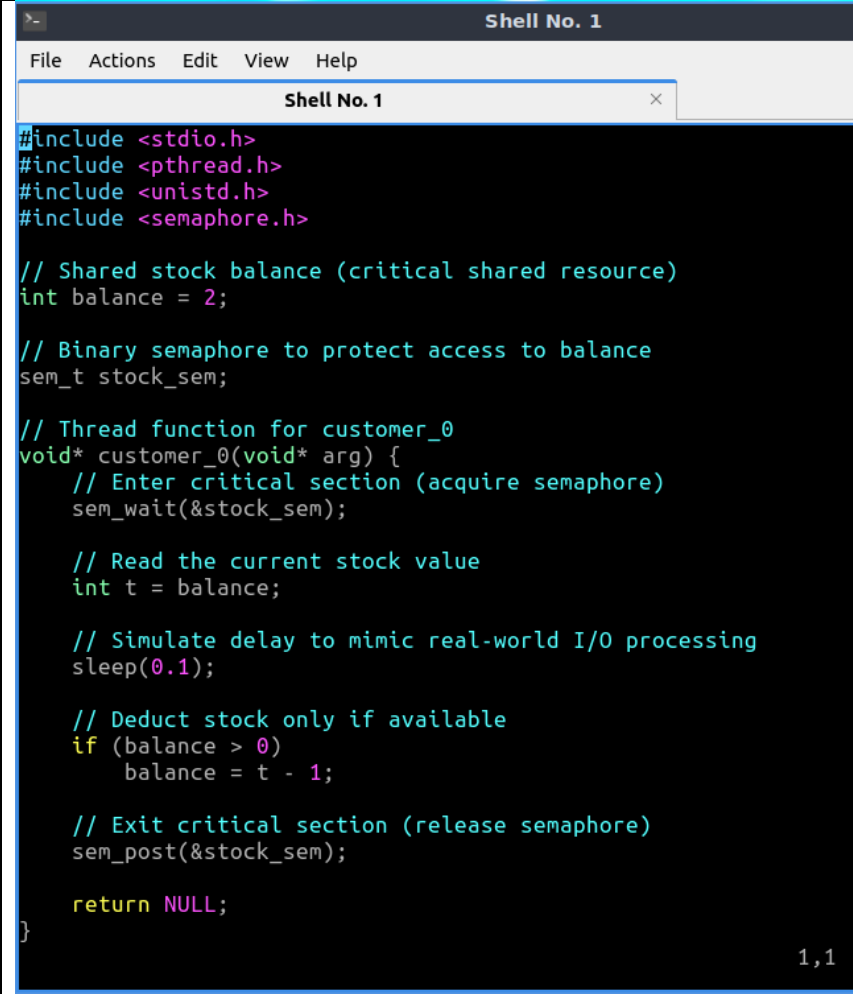
|      |     |                                                  |                                   |                                                        |
|------|-----|--------------------------------------------------|-----------------------------------|--------------------------------------------------------|
| 2.3  | car | sleep(1)                                         |                                   | The car crossing.                                      |
| 3.3  | car | sem_wait(&gate)                                  |                                   | Decrement the gate's value from 1 to 0.                |
| 3.31 | car | printf("\n The cars are crossing the railway\n") | The cars are crossing the railway |                                                        |
| 3.32 | car | sem_post(&gate)                                  |                                   | Increment the gate's value from 0 to 1.                |
| 3.33 | car | sleep(1)                                         |                                   | The car crossing.                                      |
| 4.33 | car | sem_wait(&gate)                                  |                                   | Decrement the gate's value from 1 to 0.                |
| 4.34 | car | printf("\n The cars are crossing the railway\n") | The cars are crossing the railway |                                                        |
| 4.35 | car | sem_post(&gate)                                  |                                   | Increment the gate's value from 0 to 1.                |
| 4.36 | car | sleep(1)                                         |                                   | The car crossing.                                      |
| 5.36 | car | sem_wait(&gate)                                  |                                   | Decrement the gate's value from 1 to 0.                |
| 5.37 | car | printf("\n The cars are crossing the railway\n") | The cars are crossing the railway |                                                        |
| 5.38 | car | sem_post(&gate)                                  |                                   | Increment the gate's value from 0 to 1.                |
| 5.39 | car | sleep(1)                                         |                                   | The car crossing.                                      |
| 6.2  | OS  |                                                  |                                   | The train thread has awakened from the 5 second sleep. |

After the 6.2 second, the semaphore will cause the loop iterations to repeat the same sequence that occurred from 0.16 to 6.2 second infinitely.

## **5. Conclusion**

Race condition and no synchronization is found and proven in the testing of original code. We implemented POSIX semaphore to synchronize the threads; The semaphore work by locking and unlocking access to critical session for one thread to use at a time. On top of that, during the process it also solve the race condition using a semaphore (&gate) to enforce mutual exclusion, preventing both threads to be executed simultaneously.

## Appendix A: Content of “e-commerce-solution.c”



The screenshot shows a code editor window titled "Shell No. 1" with a menu bar (File, Actions, Edit, View, Help) and a tab labeled "Shell No. 1". The code is written in C and implements a thread-safe stock management system using a semaphore. It includes headers for stdio, pthread, unistd, and semaphore. A shared stock balance is initialized to 2. A binary semaphore, stock\_sem, is used to protect access to the balance. The thread function customer\_0 acquires the semaphore, reads the current stock value, simulates a delay with sleep(0.1), deducts the stock only if it is greater than 0, and then releases the semaphore. The function returns NULL. The cursor is at line 1, column 1.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

// Shared stock balance (critical shared resource)
int balance = 2;

// Binary semaphore to protect access to balance
sem_t stock_sem;

// Thread function for customer_0
void* customer_0(void* arg) {
    // Enter critical section (acquire semaphore)
    sem_wait(&stock_sem);

    // Read the current stock value
    int t = balance;

    // Simulate delay to mimic real-world I/O processing
    sleep(0.1);

    // Deduct stock only if available
    if (balance > 0)
        balance = t - 1;

    // Exit critical section (release semaphore)
    sem_post(&stock_sem);

    return NULL;
}
```

1,1

```
// Thread function for customer_1 (same logic as customer_0)
void* customer_1(void* arg) {
    // Enter critical section (acquire semaphore)
    sem_wait(&stock_sem); // Lock

    // Read the current stock value
    int t = balance;

    // Simulate processing delay
    sleep(0.1);

    // Deduct stock if available
    if (balance > 0)
        balance = t - 1;

    // Exit critical section (release semaphore)
    sem_post(&stock_sem); // Unlock

    return NULL;
}
```

```
int main()
{
    int thread0_id = 0, thread1_id = 1;

    // Initialize binary semaphore (1 = unlocked, 0 = locked)
    sem_init(&stock_sem, 0, 1);

    pthread_t t0, t1;

    // Create both customer threads
    pthread_create(&t0, NULL, customer_0, &thread0_id);
    pthread_create(&t1, NULL, customer_1, &thread1_id);

    // Wait for both threads to finish
    pthread_join(t0, NULL);
    pthread_join(t1, NULL);

    // Display the final stock balance after both threads complete
    printf("Final stock balance: %d\n", balance);

    // Destroy the semaphore
    sem_destroy(&stock_sem);

    return 0;
}
```

## Appendix B: Content of “train-solution.c”

```
kamqingwen@osvm: ~/Downloads
GNU nano 6.2
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t gate; //Global variable, 1=road free, 0=road busy

void* train(void* arg)
{
    while(1){
        sem_wait(&gate); //Decrease gate to use road
        printf("\n The railway gate is closed\n");

        printf("\n The train is crossing\n");
        sleep(1);

        printf("\n The railway gate is opened\n");
        sem_post(&gate); //Increase gate to release road
        sleep(5);
    }
}

void* car(void* arg)
{
    while(1){
        sem_wait(&gate); //Decrease gate to occupy road
        printf("\n The cars are crossing the railway\n");
        sem_post(&gate); //Release road for cars or train
        sleep(1);
    }
}

int main()
{
    int thread0_id=0, thread1_id=1;

    sem_init(&gate, 0, 1); //Initialise semaphore to 1

    pthread_t t0,t1;

    pthread_create(&t0,NULL,car,&thread0_id);
    pthread_create(&t1,NULL,train,&thread1_id);

    pthread_join(t0,NULL);
    pthread_join(t1,NULL);

    sem_destroy(&gate); //Clean up semaphore

    return 0;
}
```

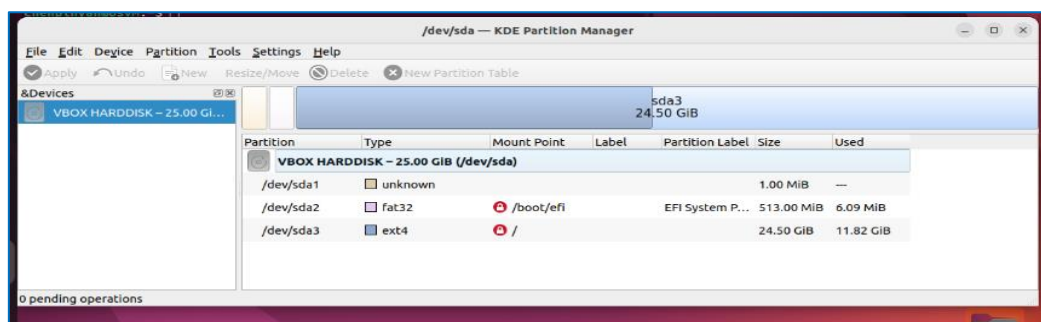
## Appendix C: Virtual Machine specs that you use to test your programs.

Member 1: Chen Pin Yan

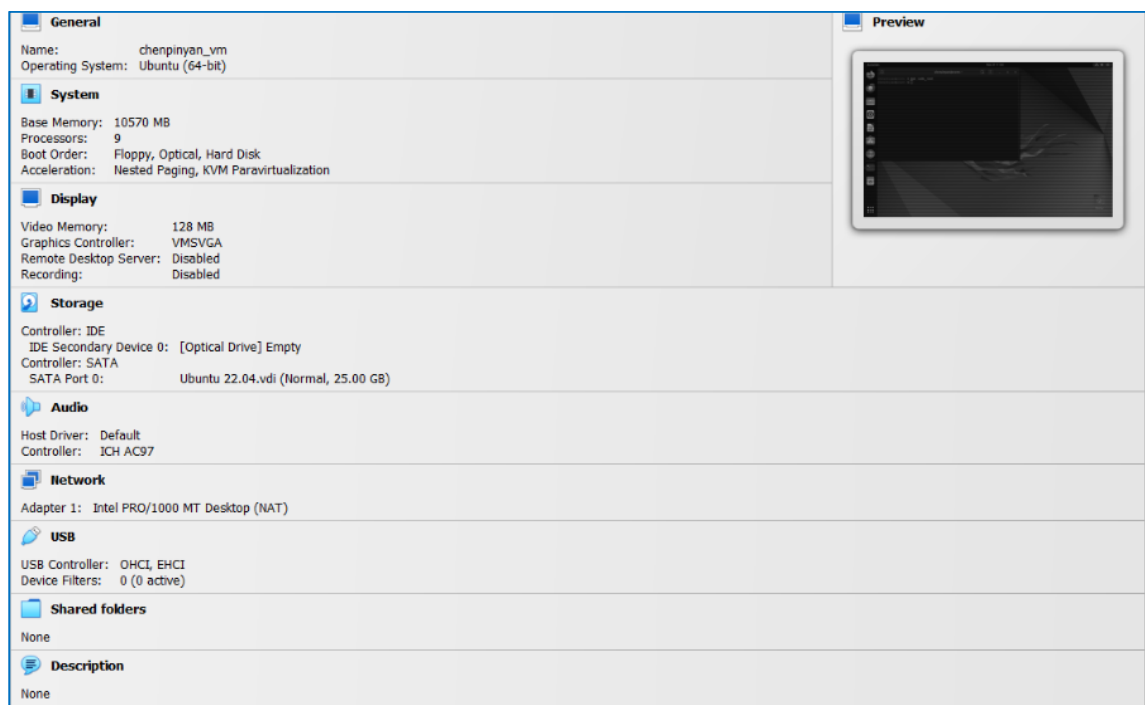
1. Screenshot of termination.

```
chenpinyan@osvm: ~  
chenpinyan@osvm:~$ man sudo_root  
chenpinyan@osvm:~$
```

2. The partition information.



3. The specs of the Virtual Machine.

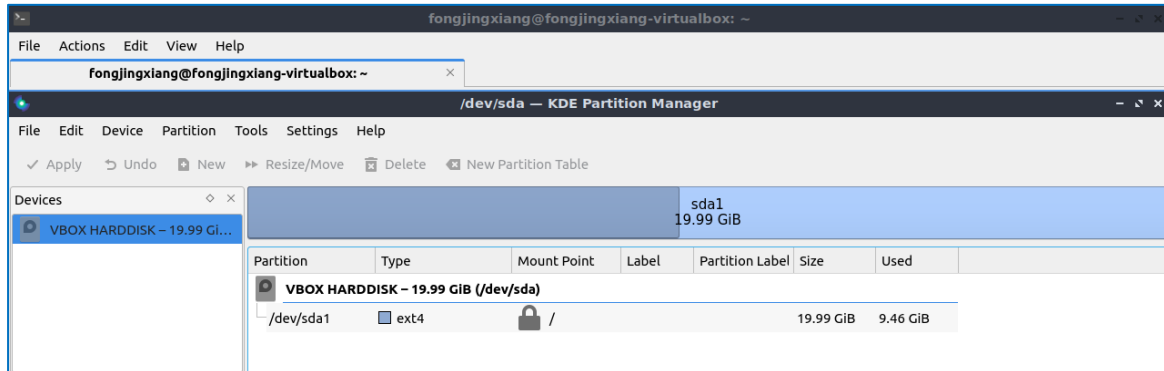


## Member 2: Fong Jing Xiang

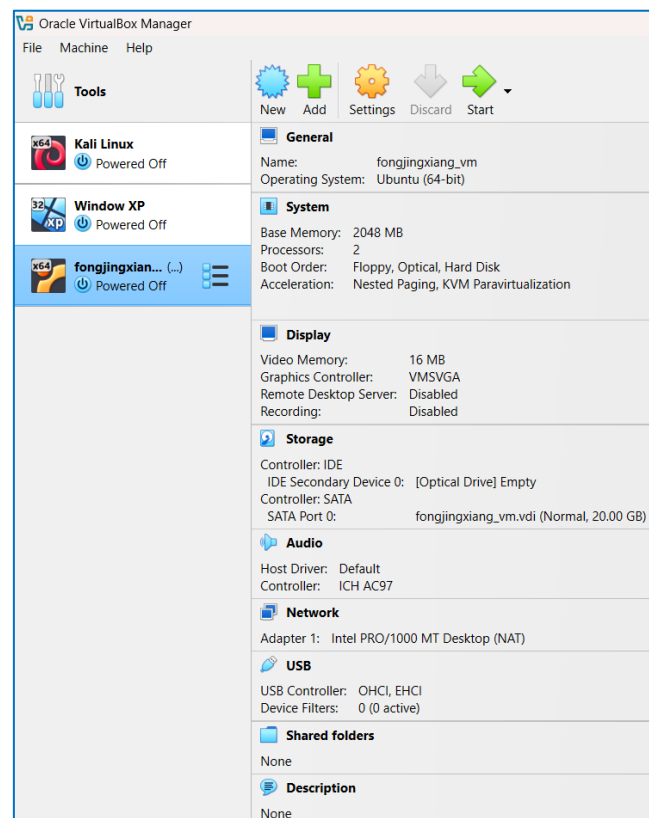
### 1. Screenshots of terminal.



### 2. The partition information.

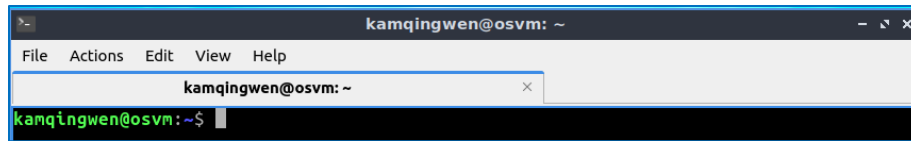


### 3. The specs of the Virtual Machine.

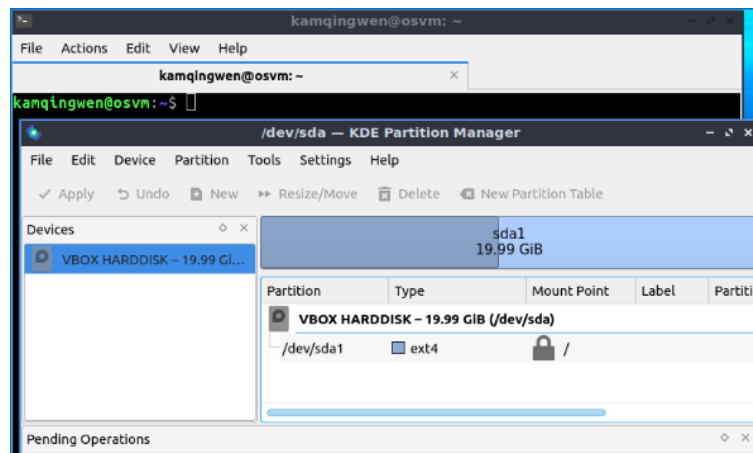


## Member 3: Kam Qing Wen

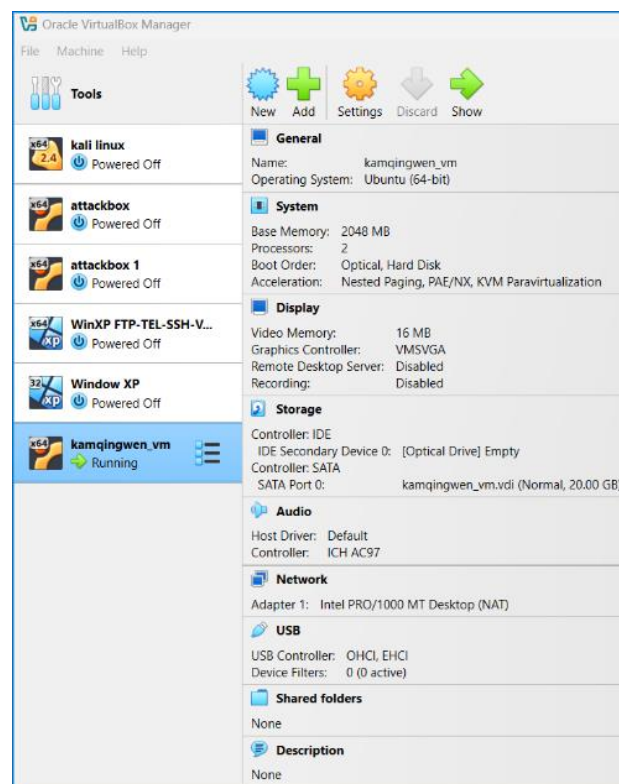
### 1. Screenshots of terminal.



### 2. The partition information.

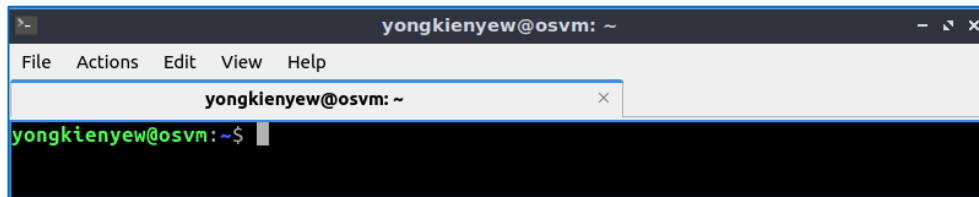


### 3. The specs of the Virtual Machine.

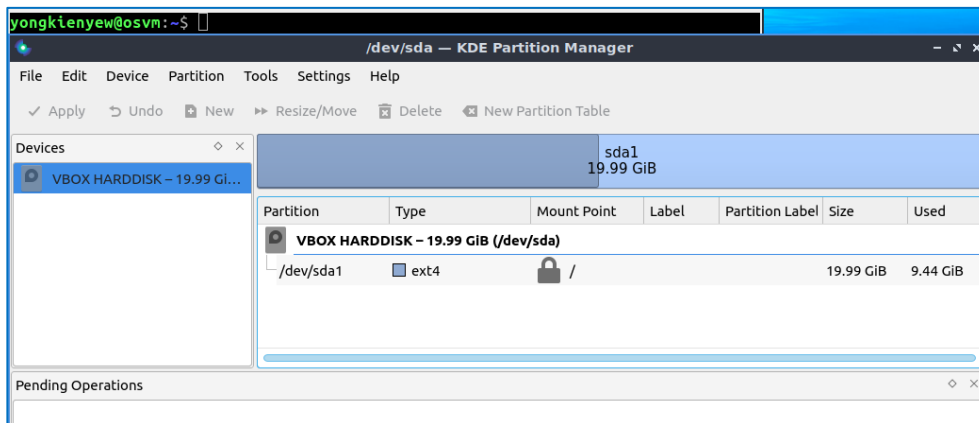


## Member 4: Yong Kien Yew

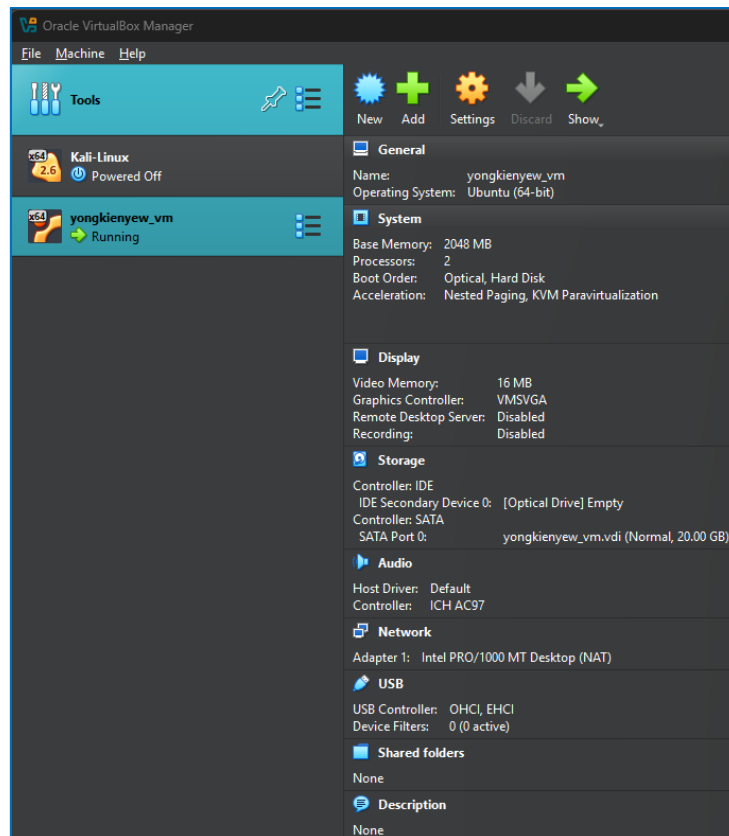
### 1. Screenshot of terminal.



### 2. The partition information.



### 3. The spec of the Virtual Machine.



## **References**

- [1] GeeksforGeeks, “How to use POSIX semaphores in C language,” GeeksforGeeks, Oct. 25, 2017. <https://www.geeksforgeeks.org/use-posix-semaphores-c> (accessed Dec. 11, 2025).
- [2] CRUCIAL DDR5. [https://shopee.com.my/CRUCIAL-DDR5-4800MHZ-RAM-DESKTOP-PC-\(UDIMM\)-LAPTOP NOTEBOOK-\(SODIMM\)-MEMORY-RAM-8GB-16GB-32GB.i.92746975.54001221948?extraParams=%7B%22display\\_model\\_id%22%3A218775637082%2C%22model\\_selection\\_logic%22%3A3%7D&sp\\_atk=6d1df262-f070-414c-9fe4-a398ac853dd5&xptdk=6d1df262-f070 414c-9fe4-a398ac853dd5](https://shopee.com.my/CRUCIAL-DDR5-4800MHZ-RAM-DESKTOP-PC-(UDIMM)-LAPTOP%20NOTEBOOK-(SODIMM)-MEMORY-RAM-8GB-16GB-32GB.i.92746975.54001221948?extraParams=%7B%22display_model_id%22%3A218775637082%2C%22model_selection_logic%22%3A3%7D&sp_atk=6d1df262-f070-414c-9fe4-a398ac853dd5&xptdk=6d1df262-f070%20414c-9fe4-a398ac853dd5)
- [3] “zentrak Rail Crossing Monitoring - voestalpine Railway Systems,” Voestalpine.com, 2025. <https://www.voestalpine.com/signaling/en/products/Rail-Crossing-Monitoring/> (accessed Dec. 11, 2025).
- [4] R. H. B. Netzer and B. P. Miller, “What are race conditions?,” ACM Letters on Programming Languages and Systems, vol. 1, no. 1, pp. 74–88, Mar. 1992, doi: <https://doi.org/10.1145/130616.130623>.